TITA Development Report

Emiliano Paradiso

August 30, 2026

Abstract

This report presents the software framework and experimental work developed for the TITA robot as part of the Master's thesis project *Acrobatic obstacle avoidance for the TITA two-wheel legged robot*. It describes the implemented control architecture and ROS 2 communication. Furthermore, it discusses several practical issues encountered during development. The complete implementation of my work is publicly available on GitHub [4]. The official GitHub repository of the DDT for documentation is [1].

Contents

1 Developed Framework **2**
- 1.1 Trajectory Generator . . . 2
- 1.2 Model Predictive Controller . . . 3
- 1.3 Whole-Body Controller . . . 3
- 1.4 State Estimation . . . 3

2 On-hardware experiments **4**
- 2.1 ROS2 Preliminaries . . . 4
- 2.2 ROS2 Communication . . . 5
- 2.3 Experiment setup . . . 6

3 Observed Issues **7**
- 3.1 Communication Latency . . . 7
- 3.2 URDF Model Consistency . . . 8
- 3.3 Wheel Velocity Estimation . . . 9
- 3.4 Control Performance . . . 9

References **10**

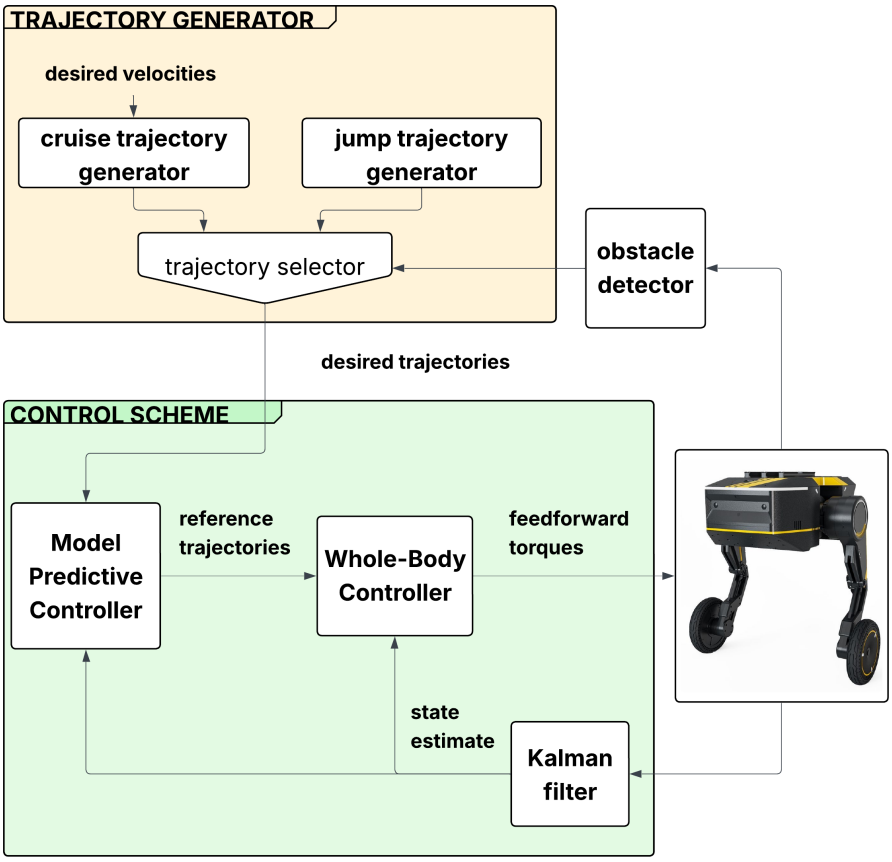


Figure 1: Overview of the control architecture implemented for the TITA robot.

1 Developed Framework

This section provides an overview of the developed control architecture, whose block diagram is shown in Figure 1. The following subsections describe the main components of the framework and their roles within the overall architecture.

A more detailed description of the modeling, formulation, and implementation of the proposed approach can be found in the Thesis [5].

1.1 Trajectory Generator

The Trajectory Generator is responsible for generating the desired trajectories of the robot's center of mass and base orientation to perform both regular locomotion and jumping maneuvers.

During nominal locomotion, the planner acts as a cruise trajectory generator. Given the desired linear and angular velocities, it integrates the motion of a unicycle model to generate the corresponding planar desired trajectory. At each control cycle, the resulting trajectory is provided to the MPC over its prediction horizon, defining the desired motion to be tracked by the controller.

When a jumping maneuver is requested, the jump trajectory generator is triggered. In this case, a detailed vertical trajectory is generated for both the CoM and the lowest points of the wheels, while preserving the previously generated planar trajectory. The jumping motion is divided into three distinct phases: *takeoff*, including the loading and push-off phases; *flight*, during which the robot follows a ballistic trajectory; and *landing*, during which the vertical motion is shaped to absorb the impact and ensure a controlled return to ground contact.

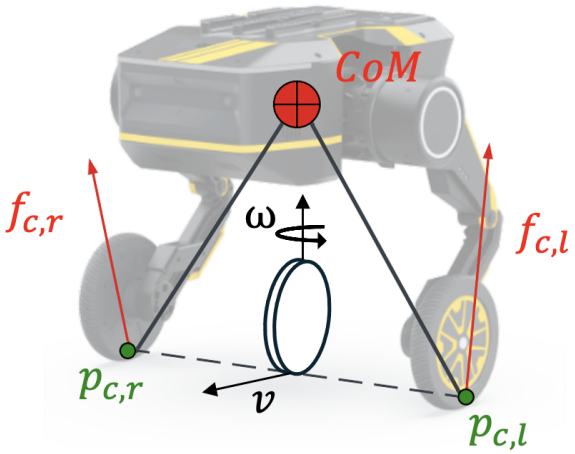


Figure 2: Illustration of the DFCIP template model.

1.2 Model Predictive Controller

Locomotion references are tracked by a nonlinear MPC built on a simplified template model, the Double Forces Constrained Inverted Pendulum (DFCIP), a variable-height inverted pendulum coupled with a unicycle-like virtual base subject to a rolling-without-slipping constraint. The model captures vertical CoM dynamics driven by two ground contact forces, enabling jump maneuvers while remaining computationally light enough for real-time solution. Contact/flight conditions are enforced through phase-dependent equality constraints, and the problem is solved with the DDP-based solver *Crocoddyl*. Since the underlying solver does not directly handle hard constraints, the required constraints are incorporated into the objective function through suitably weighted penalty terms.

A schematic representation of the simplified DFCIP model adopted for the MPC formulation is shown in Figure 2.

1.3 Whole-Body Controller

At each control cycle, the Whole-Body Controller computes the inverse dynamics by solving an optimization problem based on the full robot dynamics. The state-dependent dynamics and kinematic quantities are evaluated at the current robot state, resulting in a Quadratic Program formulation. The optimization variables are the generalized accelerations, contact forces, and joint torques.

The objective function combines weighted task-space tracking terms for the center of mass, wheel-center pose, floating-base orientation, and joint posture, together with a regularization term on the optimization variables. An additional cost is included to penalize excessive angular momentum, promoting stable motions.

The equality constraints enforce the robot dynamics and the acceleration-level rolling constraint. The latter enforces rolling without slipping while allowing the wheel to tilt.

Inequality constraints are introduced to account for the physical limitations of the robot and the contact interactions. These include joint-position/velocity and actuation limits, as well as contact-wrench feasibility constraints. In particular, the contact constraints enforce unilateral contact, restrict the tangential forces to the admissible friction cone, and prevent wheel tilting by constraining the contact wrench according to the four-point contact patch model.

Overall, the WBC provides a dynamically consistent mapping from the high-level motion references generated by the MPC to the joint torques required to execute the desired locomotion while satisfying the robot's actuation and contact constraints.

1.4 State Estimation

For deployment on the real robot, a simple Kalman filter has been implemented to estimate the floating-base position and linear velocity. In this initial implementation, the floating-base orientation and

angular velocity are directly obtained from the onboard IMU and are assumed to provide sufficiently accurate information for reconstructing the complete robot state.

The filter state comprises the floating-base linear position and velocity, together with the position of the wheel–ground contact point. The filter inputs are the linear acceleration measured by the accelerometer, the angular velocity measured by the gyroscope, and the joint velocities obtained from the encoders data. In this simplified formulation, the floating-base orientation and joint positions are treated as known parameters and are not included as stochastic states of the filter.

During the prediction step, the measured accelerations are used to propagate the floating-base velocity and position. At the same time, the position of the wheel contact point is propagated using the wheel rotational velocity. In this process, particular care is taken to distinguish the wheel velocity from the measurement of the joint encoder.

The correction step exploits the relative position between the wheel contact point and the floating base, computed through forward kinematics. This relative measurement implicitly incorporates the information provided by the floating-base orientation and joint-position measurements, being invariant from the floating base position. When a wheel is detected to be in contact with the ground, an additional constraint is imposed on the corresponding contact point, enforcing its vertical position to be zero under the assumption of a flat ground surface.

Although this formulation is intentionally simple, it provides a lightweight state-estimation framework suitable for the initial real-robot experiments, while exploiting the available IMU, encoder, and contact information to reconstruct the floating-base state.

2 On-hardware experiments

2.1 ROS2 Preliminaries

Sensor data are received through ROS 2 topics. Therefore, before conducting experiments on the robot, the ROS 2 environment must be correctly configured to establish communication between the PC and the robot. The following steps describe the required setup.

1. **Establish a network connection.**

   The PC and the robot must be connected to the same network. This can be achieved using different communication methods, such as Ethernet or Wi-Fi. A common network is required to allow the DDS middleware used by ROS2 to perform node discovery and establish communication between the two systems.

2. **Configure the ROS2 communication domain.**

   The same `ROS_DOMAIN_ID` must be configured on both the PC and the robot. In addition, communication must not be restricted to the local host. This can be configured using:

   ```
   export ROS_DOMAIN_ID=42
   export ROS_LOCAL_HOST_ONLY=0
   ```

3. **Source the ROS2 environment.**

   The appropriate ROS2 installation must be sourced on both systems.

   ```
   source /opt/ros/humble/setup.bash
   ```

   To avoid manually executing this command each time a new terminal is opened, it can be added to the `~/.bashrc` file.

4. **Verify ROS 2 discovery.**

   Once the previous steps have been completed, the connection can be verified from the PC by listing the available ROS 2 topics:

```
robot@tita:~$ ros2 topic list
/joy
/parameter_events
/rosout
/tf
/tf_static
/tita3002812/command/active/command
/tita3002812/command/manager/cmd_key
/tita3002812/command/manager/cmd_pose
/tita3002812/command/manager/cmd_twist
/tita3002812/command/manager/locomotion_cmd
/tita3002812/command/passive/command
/tita3002812/command/teleop/command
/tita3002812/command/user/command
/tita3002812/dynamic_joint_states
/tita3002812/imu_sensor_broadcaster/imu
/tita3002812/imu_sensor_broadcaster/transition_event
/tita3002812/joint_state_broadcaster/transition_event
/tita3002812/joint_states
/tita3002812/locomotion/body/fsm_mode
/tita3002812/perception/camera/image/right
/tita3002812/perception/camera/point_cloud
/tita3002812/perception/detector/angle_data
/tita3002812/perception/devices/face_point
/tita3002812/perception/devices/neck_point
/tita3002812/perception/devices/ultrawave
/tita3002812/perception/obstacle/distance_data
/tita3002812/perception/obstacle/point
/tita3002812/robot_description
/tita3002812/robot_vel
/tita3002812/system/battery/left
/tita3002812/system/battery/right
/tita3002812/system/battery_diagnostic/left
/tita3002812/system/battery_diagnostic/right
/tita3002812/system/status
```

Figure 3: Example of ROS2 topics listed.

```
ros2 topic list
```

If the configuration is correct and the ROS 2 system on the robot is running, the topics published by the robot should be visible from the PC, as shown in Figure 3.

2.2 ROS2 Communication

The controller is implemented in C++ and deployed on an Ubuntu 22.04 operating system. ROS2 Humble is used as the communication framework to interface with the robot, enabling the exchange of sensor measurements and control commands over a Wi-Fi connection. The wireless communication is established through a local network provided by a personal router. Detailed instructions for configuring the network connection are available in the official TITA development documentation, with a more comprehensive description provided in the Chinese version of the guide: https://tita-development-manual-uc.readthedocs.io/zh-cn/latest/. Sensor data are received through ROS 2 topics, while low-level motor commands are transmitted through the hardware interface provided in the DDT GitHub repository, specifically the `tita_hardware_ros2_control` package [3].

In order to enable the robot to execute joint torque commands, the hardware interface must first be launched. This requires accessing the robot's onboard computer via SSH:

```
ssh robot@<IP_ADDRESS>
```

The password is `apollo`. Once connected, navigate to the `tita_ws` workspace, source the ROS 2 environment, and launch the hardware interface with:

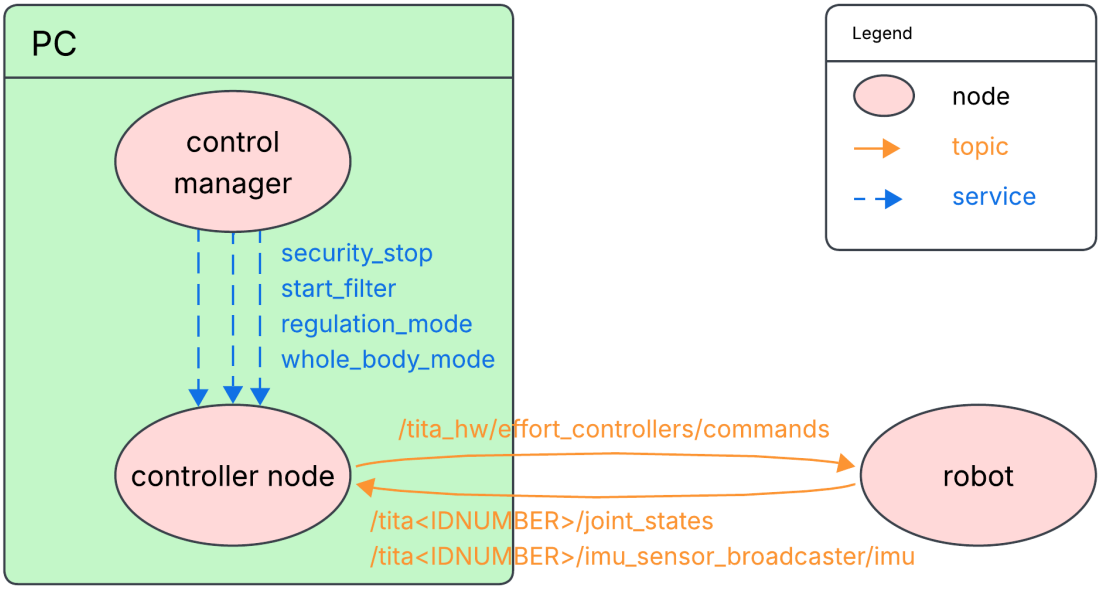


Figure 4: ROS2 nodes interactions.

```
emiliano@emiliano-IdeaPad-Slim-3-15IRH8: ~ 115x23
emiliano@emiliano-IdeaPad-Slim-3-15IRH8:~$ ros2 run tita_controller controller_manager
[INFO] [1787497733.152711730] [control_manager]: Control Manager started.
[INFO] [1787497733.152812752] [control_manager]: Press 0 → Security Stop
[INFO] [1787497733.152828095] [control_manager]: Press 1 → Regulation Mode
[INFO] [1787497733.152839648] [control_manager]: Press 2 → Start Filter
[INFO] [1787497733.152850746] [control_manager]: Press 3 → Whole Body Mode (after starting filter)
```

Figure 5: The *control manager node* running in the terminal.

```
cd tita_ws
source install/setup.bash
ros2 launch hw_bringup hw_bringup.launch.py
```

as specified in [3].

After that the communication through ROS2 is established and having launched the hardware interface a new topic will be visible as `/tita_hw/effort_controller/commands`.

2.3 Experiment setup

In order to conduct tests on the robot in a safe and controlled setting i have been developed a ROS2 package `tita_controller` consisiting of multiple nodes that can be launched to better control the flow of informations during an experiment.

Warning

Sometimes the ROS2 communication may fail and may happen that no messages are published on the topics. Before sending torque commands to the robot, make sure that the sensors are correctly published on the topics by simply checking with

```
ros2 topic echo /tita<IDNUMBER>/joint_states
```

The `tita_controller` package offers the nodes reported in Figure 4.

The *control manager node* enables the user to invoke the ROS 2 services required to manage the flow of the experiment. It provides a terminal-based interface for activating the security stop, regulation mode, state filter, and whole-body control mode. During normal operation, service calls



Figure 6: The *controller node* running in the terminal.

are performed asynchronously, while a security stop is automatically triggered when the node receives a `Ctrl+C` signal. An example of the *control manager node* running in the terminal is shown in Figure 5. The experiment is performed through a sequence of control modes. First, the **regulation_mode** service is activated, which triggers the `controller node` to publish joint torques to reach a predefined target configuration using a simple joint-level PD control law:

$$\boldsymbol{\tau} = K_P(\mathbf{q}^* - \mathbf{q}) + K_D(\dot{\mathbf{q}}^* - \dot{\mathbf{q}}). \tag{1}$$

In this mode, the controller gains are intentionally kept low to avoid making the robot excessively stiff. This allows the robot configuration to be manually adjusted, using the suspension cable, until the desired initial configuration is reached.

Once the robot has been correctly positioned, the **start_filter** service can be activated to start the state estimation filter. The filter assumes a no-slip condition at the wheels to reconstruct the robot state. Therefore, particular care must be taken during this phase to avoid wheel slippage from the initial position, as this would affect the state estimation. Before starting the whole-body controller, the robot's initial height should be verified to ensure that it is within a safe range. This can be monitored during the experiment using the Python script `plot_filter_estimate.py`, located in the `python_src` folder, which can be launched from a separate terminal. Once the state estimation is started and the robot is in a safe initial configuration, the **whole_body_mode** service can be activated. In this mode, the joint torques published by the `controller node` are computed by the whole-body controller and are combined with a local PD stabilization term:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{WBC} + K_P(\mathbf{q}^* - \mathbf{q}) + K_D(\dot{\mathbf{q}}^* - \dot{\mathbf{q}}). \tag{2}$$

The resulting torque command is then sent to the robot at the controller frequency. At any point during the experiment, if an unsafe or unexpected behavior occurs, the **security_stop** service can be invoked. This immediately commands zero joint torques, preventing further motion and avoiding damage to the robot or its surrounding environment.

Nevertheless, some automatic safety safeguards are already implemented. For example, if the commanded torque would result in a joint configuration exceeding the prescribed joint position limits at the next control step, the security stop is automatically triggered to prevent damage to the robot.

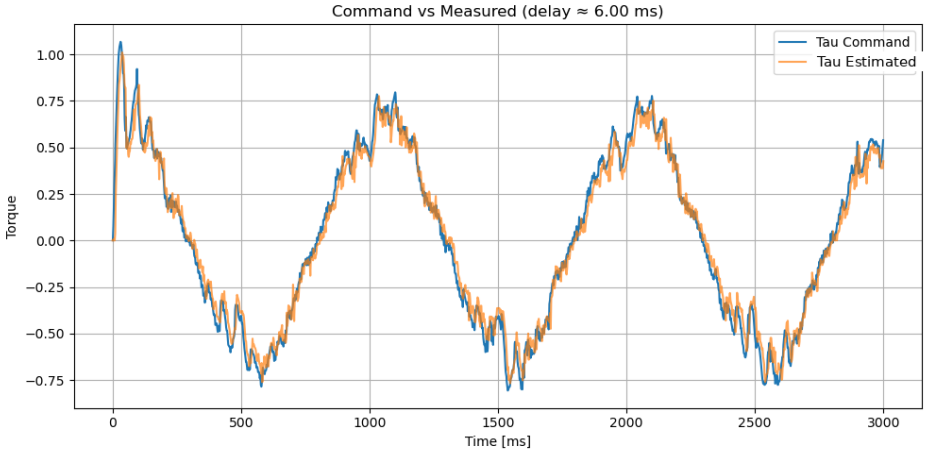
The *controller node* implements the state estimation and control algorithm that are triggered by the manager described above. It acquires IMU and joint-state measurements, estimates the robot state using a Kalman filter, and publishes joint effort commands according to the selected control mode on the `/tita_hw/effort_controller/commands` topic. The gains of the low-level PD controller can be configured through the `config.yaml` file.

For the building and installation of the `tita_controller` package, please refer to the instructions provided on the GitHub repository [4]. The most stable version of the code used in the experiments is available in the `real_controller` branch. During the final stage of the work, a new attempt to improve the control performance was carried out in the `jumping` branch (the jump remained unstable, as the robot was unable to successfully execute a jump). This development was motivated by the discovery of a method for sending low-level commands, as described in [6], which resulted in improved control performance.

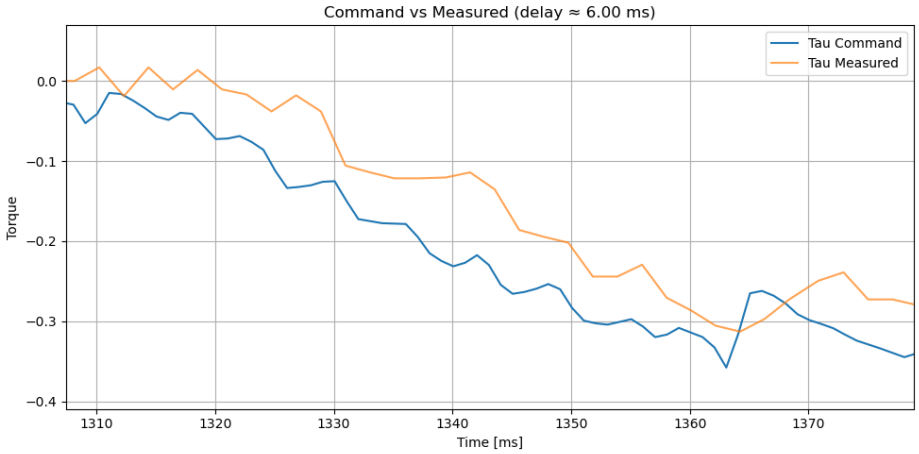
3 Observed Issues

3.1 Communication Latency

A delay of approximately 5–20 ms is observed between



(a) Comparison between commanded and estimated wheel torque.



(b) Zoomed view of the communication delay.

Figure 7: Analysis of the communication delay between commanded and measured wheel torque.

- the publication of a motor torque command, and
- the corresponding torque estimate appearing in the `/joint_state` topic.

According to the DDT support team, the reported torque is estimated from the motor current.

I also evaluated the communication through a direct Ethernet connection and implemented a simple velocity controller running onboard the robot using the `tita_robot` interface available in the `tita_hardware_ros2_control` repository [3]. Although this configuration produced the lowest latency observed during testing, a noticeable delay remained, as illustrated in Figures 7a and 7b.

3.2 URDF Model Consistency

Several URDF models are available in the official repositories, including `tita.urdf` and `titaV2.urdf`, which contain different inertial and dynamic parameters.

To verify the consistency of the robot model, I measured the physical mass of the robot using a weighing balance. The measured total mass of the robot without batteries is approximately 25.2 kg, while each battery has a mass of approximately 0.894 kg.

The developers confirmed that the most up-to-date model is provided in the `TITA_Description` repository [2].

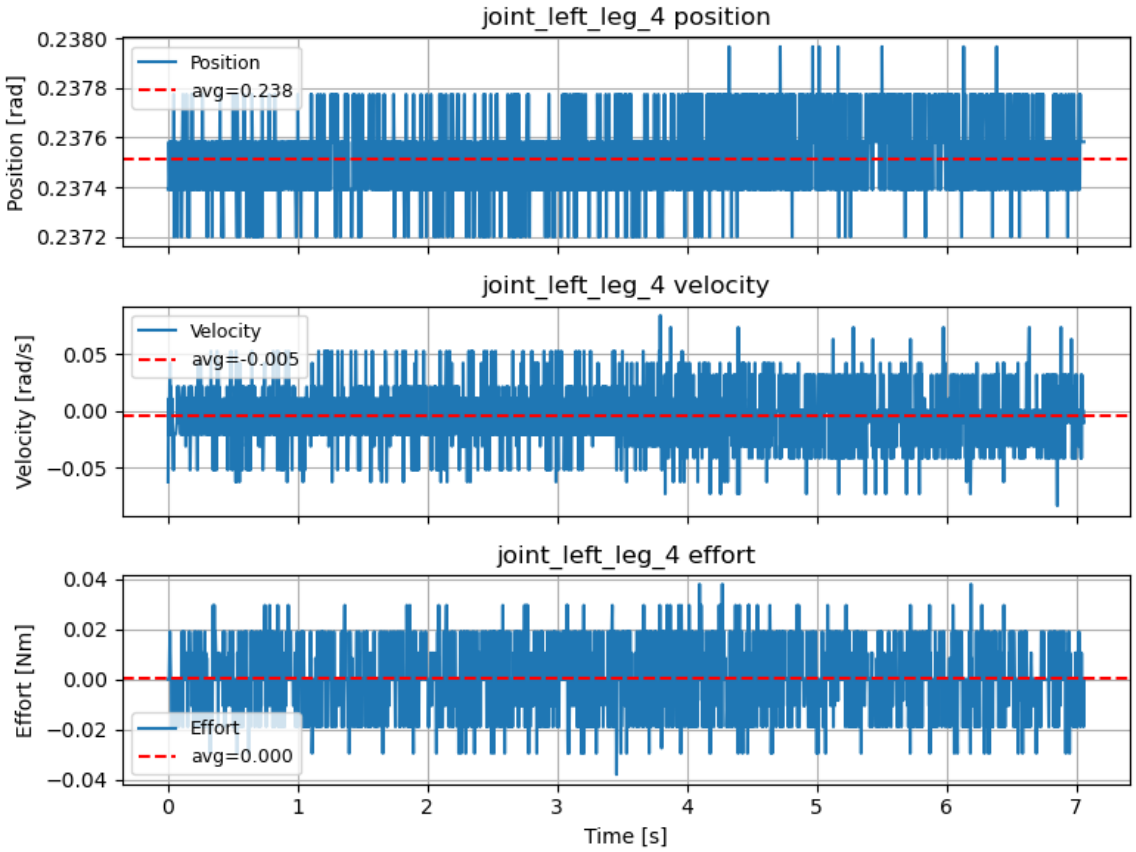


Figure 8: Wheel position, velocity, and torque obtained from the `/joint_state` topic during a stationary experiment.

3.3 Wheel Velocity Estimation

The wheel velocity estimate appears noisy and exhibits a non-zero average even when the robot is stationary with the wheels in contact with the ground, as reported in Figure 8.

According to the developers, this offset is caused by the filtering method adopted in the firmware.

3.4 Control Performance

During closed-loop balancing experiments, the following issues are observed:

- large oscillatory wheel motions while balancing,
- poor recovery after impulsive disturbances.

These issues may originate from inaccuracies in the robot model, communication latency, or both.

References

[1] DDT. Direct drive technology official repository. https://github.com/DDTRobot.

[2] DDT Robot. Tita_description. https://github.com/DDTRobot/TITA_Description.

[3] DDT Robot. tita_hardware_ros2_control. https://github.com/DDTRobot/tita_hardware_ros2_control.

[4] Emiliano Paradiso. Tita dynamic obstacle avoidance. https://github.com/Emilianogith/TITA-dynamic-obstacle-avoidance.

[5] Thesis. Master's thesis. https://github.com/Emilianogith/TITA-dynamic-obstacle-avoidance/blob/main/assets/paradiso_1940454.pdf.

[6] Haizhou Zhao, Lei Yu, Siying Qin, Gumin Jin, and Yuqing Chen. Design and control of a bio-inspired wheeled bipedal robot. *arXiv preprint arXiv:2308.13205*, 2023.